

Sesiunea 4: JS DOM Manipulation

Chapter 1: Connecting JavaScript to the Page (DOM Element Selection)

Learning Objectives

1.1 document - The Bridge

1.2 The Ultimate Tool: document.querySelector()

1.3 querySelectorAll() - When we want the full list

1.4 Did you know...?

1.5 Practical Exercises

1.6 Exercise Solutions

1.7 🤖 Engineer Mindset in the AI Era: Your Place in the Market

Chapter 2: Bringing elements to life - Event Listeners (addEventListener)

Learning Objectives

2.1 What is an Event?

2.2 Syntax: addEventListener

2.3 Common event types for QA

2.4 Did you know...?

2.5 Practical Exercise (End-to-End Laboratory)

2.6 How does this translate to a QA's work?

2.7 🤖 Engineer Mindset in the AI Era: Your Place in the Market

Chapter 3: Dynamic Addition and Deletion of DOM Elements

Learning Objectives

3.1 The Birth of an Element: document.createElement()

3.2 Insertion into the Page: appendChild()

3.3 The Lifecycle of a Dynamic Element

3.4 Destroying an Element: remove()

3.5 Practical Workshop: Functional Task Tracker!

3.6 🤖 Engineer Mindset in the AI Era: "The moving needle in the haystack"

Chapter 4: Advanced Concepts (Bonus) - Latency and Event Propagation

Learning Objectives

4.1 Automation's Enemy Number 1: Latency (setTimeout)

4.2 How does Latency destroy QA tests?

4.3 The secret of interfaces: Event Bubbling

4.4 Stopping propagation (stopPropagation)

4.5 🤖 Engineer Mindset in the AI Era

Chapter 5: Session 4 Assignment & Extra Practice (Towards Senior Level)

5.1 Homework Assignment (Session 4 Project)

5.2 Bonus Challenge (Optional): Smart Button Disabling

5.3 🗝️ Solution Keys (Answers & Explanations)

Chapter 1: Connecting JavaScript to the Page (DOM Element Selection)

Learning Objectives



- Understand the role of the global document object as a bridge between your JS code and the visual HTML structure.
- Use the `document.querySelector()` command to find specific elements on the page, based on the CSS selectors learned previously.
- Differentiate between selecting a single element and selecting a list of elements with `document.querySelectorAll()`.
- Discover the direct parallel between native JavaScript selection and the commands you will use later in Playwright (e.g., `page.locator()`).

1.1 document - The Bridge

Up until now (in Session 3), we have run JavaScript code in the console to do mathematical calculations, work with variables, and use functions. But our code was somewhat blind. It knew absolutely nothing about the buttons or texts in our HTML file.

When the browser loads your HTML page, it transforms it into a hidden data structure called the **DOM (Document Object Model)**. The good news? The browser automatically creates a global variable in JavaScript called **document**.

This variable is the "universal key" to your application. It represents and contains the entire web page!

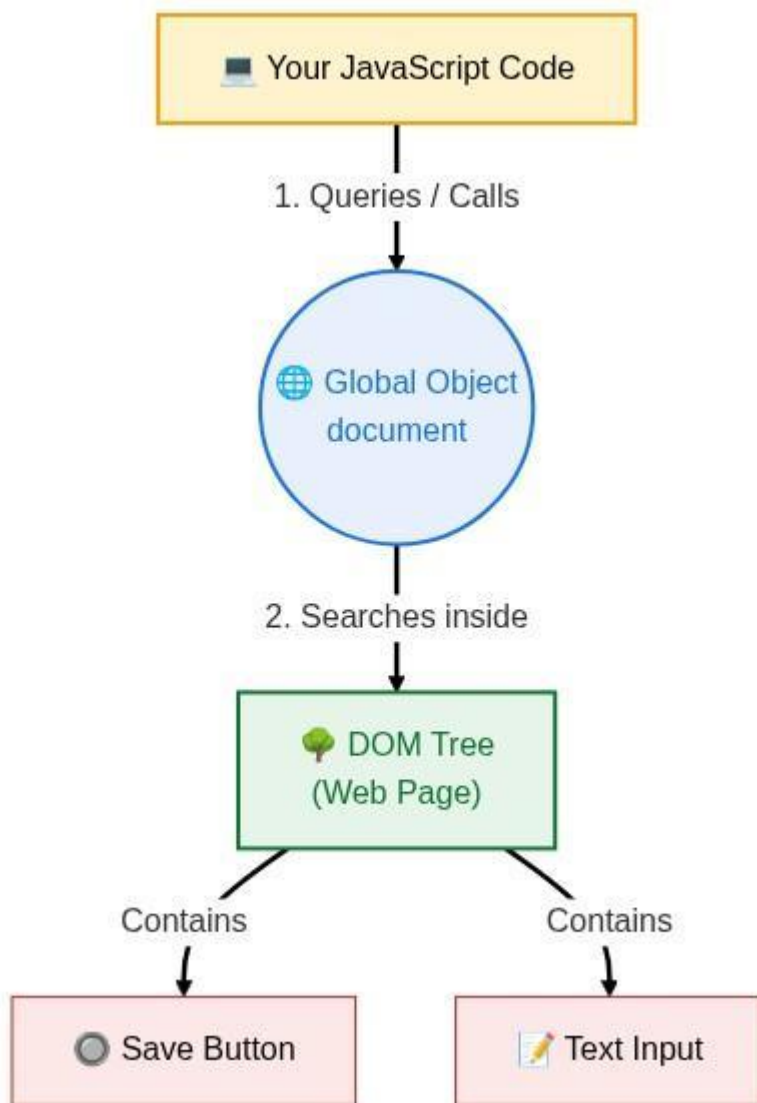


Fig. 1 - The diagram illustrates the interaction architecture between a JavaScript script and web page elements. **JavaScript code** accesses the HTML structure via the **global document object**, which serves as the programming interface. It queries the **DOM tree** to identify, read, and dynamically manipulate the final interactive elements, such as the **Save Button** and **Text Input**.

1.2 The Ultimate Tool: document.querySelector()

How do we tell JavaScript to "grab" a button so we can interact with it later? By using the exact "CSS Selectors" you learned in Session 2 (classes, IDs, attributes)!

The most important and modern method is `document.querySelector('css_selector')`. This searches the page (from top to bottom) and returns the **first element** that matches your description.

Usage examples:

```
// Find an element by ID (Recommended)
const addButton = document.querySelector('#add-task-btn');

// Find an element by Class (Will only bring the first one found!)
const firstTask = document.querySelector('.task-item');

// Find an element by a Test Attribute (QA Automation Style)
const deleteButton = document.querySelector('[data-testid="delete-btn"]');

// Print what we found in the console to be sure
console.log(addButton);
```

QA Analogy: `querySelector` is like a tracking dog. You let it smell a piece of clothing (the CSS selector), and it runs through the whole building (DOM) and brings back the first person that matches.

1.3 querySelectorAll() - When we want the full list

What happens if you want to count how many tasks you have in your application? If you use `querySelector`, you will only get the first task, ignoring the rest.

To get a complete list (very similar to an Array), we use `document.querySelectorAll()`.

```
// Bring ALL elements that have the .task-item class
const allTasksList = document.querySelectorAll('.task-item');

// Now we can use the .length property (just like with Arrays)
console.log("We have a total number of tasks: " + allTasksList.length);
```

This concept is essential for future tests. In Playwright, when you check if exactly 3 products were added to a shopping cart, the framework executes a very similar mechanism in the background to count the elements in the DOM!

1.4 Did you know...?



- **\$0 in DevTools:** There is a secret trick in the Chrome Console. If you go to the *Elements* tab, click on any HTML code, then switch to the *Console* tab, type simply \$0 and press Enter, JavaScript will instantly return that element without you having to write any `querySelector`! It is brilliant for rapid debugging.
- **Legacy Methods:** On older websites (or in tutorials from 2010), you will often see methods like `document.getElementById('id')` or `document.getElementsByClassName('class')`. Although they work perfectly and are slightly faster to execute, the modern industry has standardized the use of `querySelector` because it allows you to write much more complex and chained queries (e.g., `.container > button:first-child`) using a single, unified command.

1.5 Practical Exercises

Open the *index.html* file of your "Task Tracker" app in the browser, press *F12* and go to the *Console* tab.

Mission 1:

Use `document.querySelector` to find the text field where the user types the name of a task (the text input). Save it in a constant named `taskInput` and print it in the console using `console.log`.

Mission 2:

Use `document.querySelectorAll` to find absolutely all `<button>` tags on your page. How many buttons do you have in total in the application right now?

1.6 Exercise Solutions

Solution Mission 1:

```
// Assuming your input has the id "task-name" or the class "task-input"
const taskInput = document.querySelector('#task-name');
console.log(taskInput);
```

✓ Solution Mission 2:

```
const allButtons = document.querySelectorAll('button');
console.log(allButtons.length); // Will print a number, ex: 3
```

1.7 🧑🏻‍💻 Engineer Mindset in the AI Era: Your Place in the Market

🔑 QA vs. AI: The Transition to Automation Thinking

- **What AI does well:** If you ask an AI (e.g., ChatGPT, GitHub Copilot) to write code to manipulate a page, it will generate code instantly.
- **The Trap (Where AI fails):** AI was trained on millions of lines of code that are over a decade old. If you are not specific, AI will often generate selectors based on `document.getElementsByTagName` or tie itself to design-specific CSS classes (e.g., `.bg-blue-500`) to find elements. This approach generates "fragile" scripts that will break as soon as the UI undergoes minor visual modifications.
- **How a QA engineer thinks:** A QA engineer knows that testability is king. They will write (or instruct the AI to write) selections based exclusively on `querySelector` using stable test attributes, like `[data-testid="submit-btn"]`. Why? Because when you transition to Playwright (Module 4), the native command `document.querySelector('[data-testid="submit-btn"]')` will architecturally translate 1:1 into `page.getByTestId('submit-btn')`.
- **Why you will stay relevant in the market:** A simple executor blindly copies the code provided by AI. You, on the other hand, act as a "Testing Architect". By enforcing a specific element selection methodology even in basic JavaScript, you ensure that the application and the tests will "speak" the exact same language, drastically reducing long-term maintenance time for the entire team.

Chapter 2: Bringing elements to life

- Event Listeners (addEventListener)

Learning Objectives



- Understand what an Event is in the context of a web browser.
- Use the `addEventListener` method to make HTML elements react to user actions.
- Work with *Callback* functions (Arrow Functions) that execute only at the right moment.
- Correlate native JavaScript events with the testing actions you will execute later with Playwright.

2.1 What is an Event?

As soon as the web page has loaded, the browser sits and "listens" quietly. When the user moves the mouse, clicks, presses a key, or scrolls, the browser generates an **Event**.

Without JavaScript code to react to these events, your clicks would hit HTML elements with absolutely nothing happening (except for native elements, like `<a>` links).

This is where the **Event Listener** comes into play. It acts like a security guard you place next to a button, telling them: *"Stand here and watch this button. When someone clicks on it, trigger the alarm (my function) immediately!"*

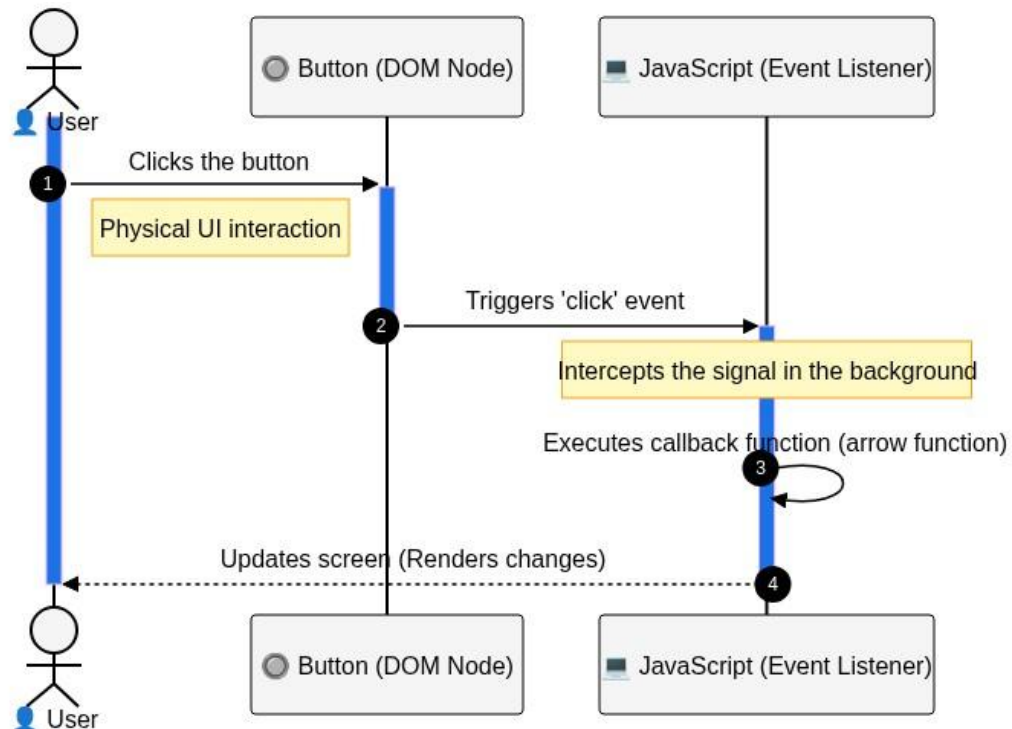


Fig. 2 - The sequence diagram illustrates the asynchronous workflow of event handling in the browser. The process begins with the **user interaction** (click), at which point the **DOM element** captures the action and emits a signal to **JavaScript**. The script asynchronously intercepts the event via the *Event Listener*, executes the *callback* function, and instantly reflects the resulting changes in the graphical user interface.

2.2 Syntax: addEventListener

To add this interactivity, we use a method (a command) attached directly to the element we selected from the DOM in the previous chapter. This method needs two "ingredients" (parameters):

- **The event name** (a String, e.g., 'click').
- **What should happen** (an Arrow Function, also called a *Callback* function, containing the code to be executed).

```

// Step 1: Find the element on the page
const saveButton = document.querySelector('#add-task-btn');

// Step 2: Attach a "listener" to it
saveButton.addEventListener('click', () => {
    // Everything written between these curly braces will execute ONLY
    when clicked!
})

```

```
    console.log("You clicked the save button!");  
  });
```

Important note: The Arrow Function above does not execute immediately when JavaScript reads the file. It sits "waiting" in memory, sometimes endlessly, until the user performs the click action.

2.3 Common event types for QA

In automation, you will simulate human actions. Here are the most used events that developers listen for and that you will trigger with your robot:

- 'click': Fired upon pressing a button or a link.
- 'input': Fired every time the value in a text field (<input>) changes (on every typed letter). In Playwright, the equivalent action that triggers this event is `page.fill()`.
- 'change': Similar to input, but usually fired at the end (when the user leaves the text field) or when they select an option from a drop-down menu (<select>).
- 'keydown' / 'keyup': Fired when a physical key is pressed (e.g., pressing the Enter key to submit a form).

2.4 Did you know...?



- **Playwright doesn't magically "click":** When you tell Playwright `page.click('#btn')`, it doesn't send an invisible finger to the screen. "Under the hood," it sends a signal directly to the browser engine to artificially emit (dispatch) exactly this 'click' event. If the developer forgot to put `addEventListener` on the button, your Playwright test will "click" successfully (it won't throw an error), but nothing will happen in the app!
- **Beware of Inline attributes (Anti-pattern):** You might have seen old HTML code that looks like this: `<button onclick="saveData()">`. This is called an "Inline Event Handler". It is an outdated practice avoided today because it mixes structure (HTML) with logic (JavaScript), violating the *Separation of Concerns* principle.

2.5 Practical Exercise (End-to-End Laboratory)

We will visually manipulate the DOM of our "Task Tracker" application. Let's write a script that



catches the "Save Task" button, "listens" for a click, and instantly changes its text and color!

Step 1: The full HTML Code (index.html)

```
section {
  margin-bottom: 25px;
}

h2 {
  font-size: 1.2rem;
  margin-bottom: 15px;
  color: #34495e;
  border-bottom: 2px solid #ecf0f1;
  padding-bottom: 5px;
}

form {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

label {
  font-weight: 600;
  font-size: 0.95rem;
}

input[type="text"] {
  padding: 10px;
  font-size: 1rem;
  border: 1px solid #ccc;
  border-radius: 4px;
  outline: none;
  transition: border-color 0.3s;
}

input[type="text"]:focus {
  border-color: #007bff;
}

button {
  padding: 12px 20px;
  font-size: 1rem;
  cursor: pointer;
  border: none;
  border-radius: 4px;
  background-color: #007bff;
  color: white;
  font-weight: bold;
}
```

```

        transition: background-color 0.3s, transform 0.1s;
        margin-top: 5px;
    }

    button:hover {
        background-color: #0056b3;
    }

    button:active {
        transform: scale(0.98);
    }

    ul {
        list-style-type: none;
    }

    /* Pre-styled class for when JS injects tasks */
    .task-item {
        background: #ecf0f1;
        padding: 10px;
        margin-bottom: 8px;
        border-radius: 4px;
        display: flex;
        justify-content: space-between;
        align-items: center;
    }
</style>
</head>
<body>

    <header>
        <h1>Task Tracker Pro - QA Edition</h1>
    </header>

    <main>
        <!-- Task addition section -->
        <section class="task-input-section">
            <h2>Add a New Task</h2>
            <form id="add-task-form">
                <label for="task-name">Task Name:</label>
                <input type="text" id="task-name" data-testid="input-
task-name" placeholder="e.g., Write E2E scenarios" required>

                <!-- The target button for our exercise -->
                <button type="submit" id="add-task-btn" data-

```

```

testid="submit-new-task">Save Task</button>
    </form>
</section>

    <section class="task-list-section">
        <h2>Active Tasks</h2>
        <!-- This is the CRITICAL element your JavaScript is looking
for! -->
        <ul id="active-tasks-list">
            <!-- The new <li> elements you add will dynamically
appear here -->
        </ul>
    </section>
</main>

    <!-- Link to the JavaScript file -->
    <script src="app.js"></script>

</body>
</html>

```

Step 2: The JavaScript Code (app.js)

Write the following code in your JS file and test it in the browser!

```

// STEP 1: Find the element on the page (Selection)
const saveButton = document.querySelector('#add-task-btn');

// STEP 2: "Listen" for the user's interaction
// The 'event' parameter (or 'e') catches the details of the physical
click action
saveButton.addEventListener('click', (event) => {

    // CRITICAL TRICK: We stop the native behavior of the form!
    // If we don't put this, the page instantly refreshes and we lose
the state.
    event.preventDefault();

    // STEP 3: Live DOM Modification (Change Text)
    saveButton.textContent = "Task Saved! 

```

```
// Report in the console for QA
console.log("Interaction detected! Function stopped the refresh and
modified the DOM.");
});
```

2.6 How does this translate to a QA's work?

The exercise above represents exactly what a Front-End developer does. Your role as an automation engineer is to verify if this `addEventListener` worked. Later in the course, your Playwright test for this scenario will look exactly like this:

```
// This is how your E2E test will look in Module 4!
await page.getByTestId('submit-new-task').click();
await expect(page.getByTestId('submit-new-task')).toHaveText('Task
Saved! ☒');
```

2.7 Engineer Mindset in the AI Era: Your Place in the Market

QA vs. AI: The "onClick" Trap and Testability

- **What AI does well:** When you ask a programming assistant (e.g., GitHub Copilot) to add interactivity to a button in a project, it will do it quickly.
- **The Trap (Where AI fails):** Sometimes, especially on prototype projects, a lazy AI will directly alter your HTML file and add the `<button onclick="submitForm()">` attribute (Inline events), instead of using the modern architecture with `addEventListener` in the JS file.
- **How a QA engineer thinks:** The engineer immediately looks at the code (Code Review) and rejects this approach. They know that logic mixed with the UI makes the application much harder to test. If an event listener is cleanly attached in JavaScript, you can "mock" (simulate/intercept) the `submitForm()` function much easier during Component tests. The engineer will demand code refactoring to keep the app clean and up to Enterprise standards.
- **Why you will stay relevant in the market:** Your value is not in quickly writing a function that "clicks" (AI does that in a second). Your value is in defending **architecture** and **best practices**. Applications that bypass standards eventually become an unmaintainable "swamp". As a QA, you are the voice of quality starting right from the

source code level.

Chapter 3: Dynamic Addition and

Deletion of DOM Elements

Learning Objectives



- Understand the difference between an element created in JavaScript memory and a visible element on the screen.
- Use `document.createElement()` to generate new HTML elements from code.
- Inject new elements into the page structure using `appendChild()` or `append()`.
- Remove elements from the page using the `remove()` method.
- Correlate these concepts with real challenges in automated testing (elements appearing with a delay).

3.1 The Birth of an Element: `document.createElement()`

In modern web applications (written in React or Angular), the initial HTML is often almost empty. The interface is built "on the fly" (dynamically) by JavaScript, as data comes from the server or the user interacts with the page.

To create a new element directly from JavaScript, we use the `document.createElement('tag_name')` command.

```
// 1. We create a list element <li> in the browser's MEMORY
const newTask = document.createElement('li');

// 2. We configure it (give it text, classes, attributes)
newTask.textContent = "Learn Playwright";
newTask.className = "task-item";
```

Warning: At this moment, the `newTask` element only exists in JavaScript's temporary memory. It is **NOT** visible on the screen because it has not been attached to the DOM's "Family Tree".

3.2 Insertion into the Page: `appendChild()`

To make the element visible, we must hook it to a "Parent" already existing in the HTML. Think

of this like assembling a Lego set: first, you build the new piece in your hand (`createElement`), then you plug it into the baseplate (`appendChild`).

```
// We find the parent on the page (e.g., our task list <ul>)
const taskList = document.querySelector('#active-tasks-list');

// We attach the new element at the end of the child list
taskList.appendChild(newTask);
```

Now, the magic has happened! The element appeared on the screen instantly, without the page refreshing itself.

3.3 The Lifecycle of a Dynamic Element

Here is the visualization of the complete process, from idea to display on the screen:

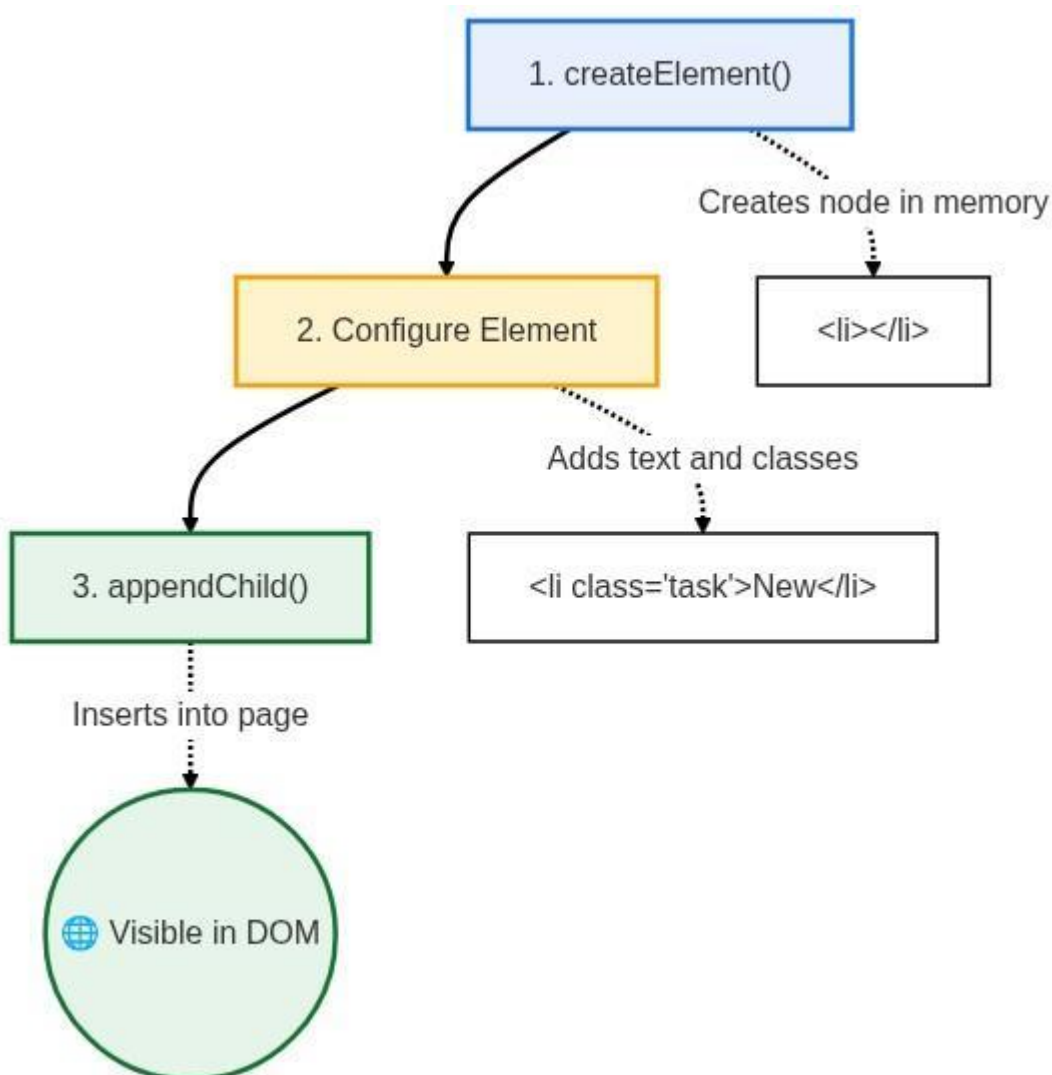


Fig. 3 - The diagram shows the lifecycle of dynamically creating and inserting an element into

the UI using JavaScript. The process is structured into three consecutive steps: **creating the node in RAM memory** (`createElement`), **configuring properties** (populating text content and attaching CSS classes for styling), and **appending it to the web page hierarchy** (`appendChild`), making it instantly visible to the user.

3.4 Destroying an Element: `remove()`

Deleting an element is much simpler than creating it. If you have the reference to that element, you simply call the `.remove()` method.

```
// We find the first delete button on the page
const deleteBtn = document.querySelector('.delete-btn');

// We remove it completely from the DOM
deleteBtn.remove();
```

QA Note: In automation, verifying deletion is a critical assertion (e.g., `expect(locator).not.toBeVisible()`).

3.5 Practical Workshop: Functional Task Tracker!

Let's unite everything we've learned in Session 4 (Selection + Events + Dynamic DOM). We will make your form actually add real tasks to the list!

Open your **app.js** file and replace the old code with this one. Make sure you have the `<ul id="active-tasks-list"></ul>` element in your HTML. Then test the app in the browser!

```
// 1. Select the elements we need
const form = document.querySelector('#add-task-form');
const taskInput = document.querySelector('#task-name');
const taskList = document.querySelector('#active-tasks-list');

// 2. Listen for the 'submit' event of the form
form.addEventListener('submit', (event) => {
  // Stop the page reload
  event.preventDefault();

  // Extract the value (text) written by the user in the input
```

```
const newText = taskInput.value;

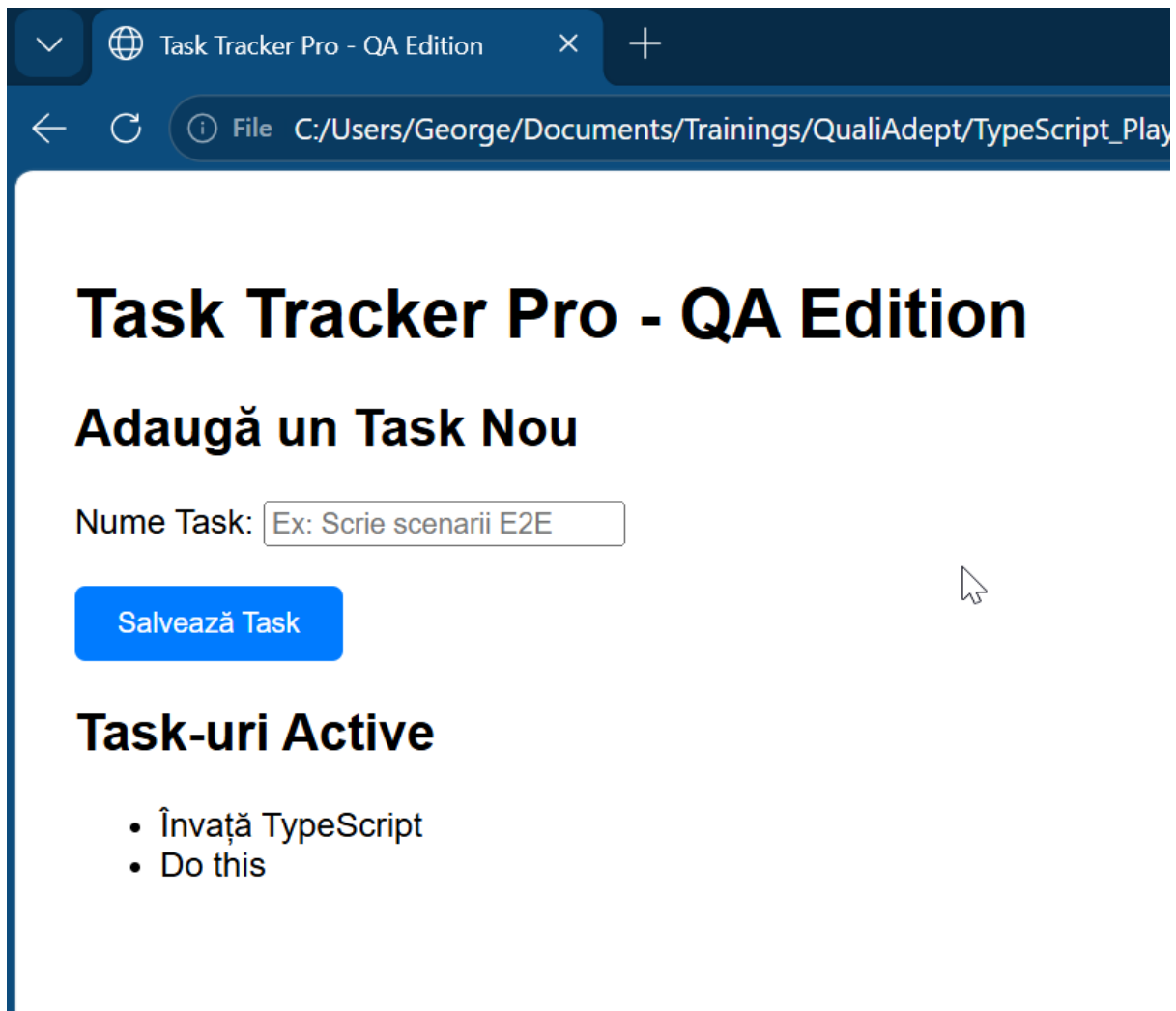
// 3. Create a new <li> element
const newLi = document.createElement('li');
newLi.className = 'task-item';
newLi.textContent = newText;

// 4. Inject it into the DOM (inside the <ul>)
taskList.appendChild(newLi);

// 5. Clear the input to make room for the next task
taskInput.value = '';

console.log("Task successfully added: " + newText);
});
```

Test it! Type "Learn TypeScript" into the box and press "Save Task". You will see the list grow live, right before your eyes! Your application is taking real shape.



3.6 🤖 Engineer Mindset in the AI Era: "The moving needle in the haystack"



🔒 QA vs. AI: The Illusion of a Static DOM

- **What AI does well:** LLM-based assistants perfectly understand how to write a function that clicks an element. They logically assume: *Find element X -> Execute action Y*.
- **The Trap (Where AI fails):** AI approaches a web page like a PDF document (something static and unchanging). But you've just seen how elements are created and deleted in milliseconds (`createElement`, `remove`). If an AI-generated test tries to validate a task's text immediately after clicking "Save", the test might suddenly fail

(Flaky Test). Why? Because the AI didn't account for the 50 milliseconds it takes for JavaScript to execute and inject the element into the DOM with `appendChild`. The script was too fast for the interface!

- **How a QA engineer thinks:** The engineer knows the DOM is a living organism. When a QA engineer writes an E2E test (or corrects AI code), they don't assume the element is there. They use methods with **Active Waiting (Auto-Waiting)**. In Playwright, you will write commands like: *"Wait until the element with text X is attached to the DOM (State: attached), and only then validate it"*.
- **Why you will stay relevant in the market:** The difference between a mediocre tester and a Senior Automation Engineer is the ability to stabilize tests. By understanding the exact mechanics of how Frontend frameworks add elements to the page, you will know why your tests fail and, more importantly, how to write assertions resilient to browser rendering latencies.

Chapter 4: Advanced Concepts (Bonus) - Latency and Event

Propagation

Learning Objectives



- Understand the latency (delay) phenomenon in web applications using `setTimeout`.
- Realize how latency causes "Flaky" (unstable) tests and why strict assertions fail.
- Discover the "Event Bubbling" concept in the DOM.
- Correlate these advanced JavaScript mechanisms with Playwright's Auto-Waiting features.

4.1 Automation's Enemy Number 1: Latency (`setTimeout`)

In our examples so far, when you clicked a button, the text or the new element appeared instantly (in zero milliseconds). In the real world, things aren't like that. When you add a product to the cart on an online store, the browser has to send the data to the server, wait for confirmation, and only then display the success message on the screen. This journey can take anywhere from 100ms to a few seconds.

To simulate this delay (latency) in JavaScript, we use the `setTimeout()` function.

```
const orderButton = document.querySelector('#place-order');

orderButton.addEventListener('click', () => {
  console.log("Order is processing...");

  // We simulate the server's response with a 3-second delay (3000 milliseconds)
  setTimeout(() => {
    orderButton.textContent = "Order Placed!";
    orderButton.style.backgroundColor = "green";
    console.log("UI updated after 3 seconds.");
  }, 3000);
});
```

4.2 How does Latency destroy QA tests?

If you wrote a classic E2E test (old style, like Selenium in its early days) for the code above, your test would look something like this:

- Click on #place-order
- Validate that the button's text is "Order Placed!" -> **TEST FAILED! ✖**

Why did it fail? Because your robot executes step 2 in exactly 5 milliseconds after step 1. The button didn't have time to turn green, because the server (simulated by `setTimeout`) needed 3000 milliseconds! This is the famous **Flaky Test** (a test that fails and passes seemingly randomly, depending on internet speed).

Playwright's Solution: When we reach the Playwright module, our assertions will look like this:

```
await expect(orderButton).toHaveText("Order Placed!");
```

The key phrase here is *Auto-Waiting*. Playwright doesn't just check once and fail. It checks dozens of times per second, waiting (up to a default maximum timeout of 5 seconds) for that `setTimeout` in JavaScript to finish its job and modify the DOM!

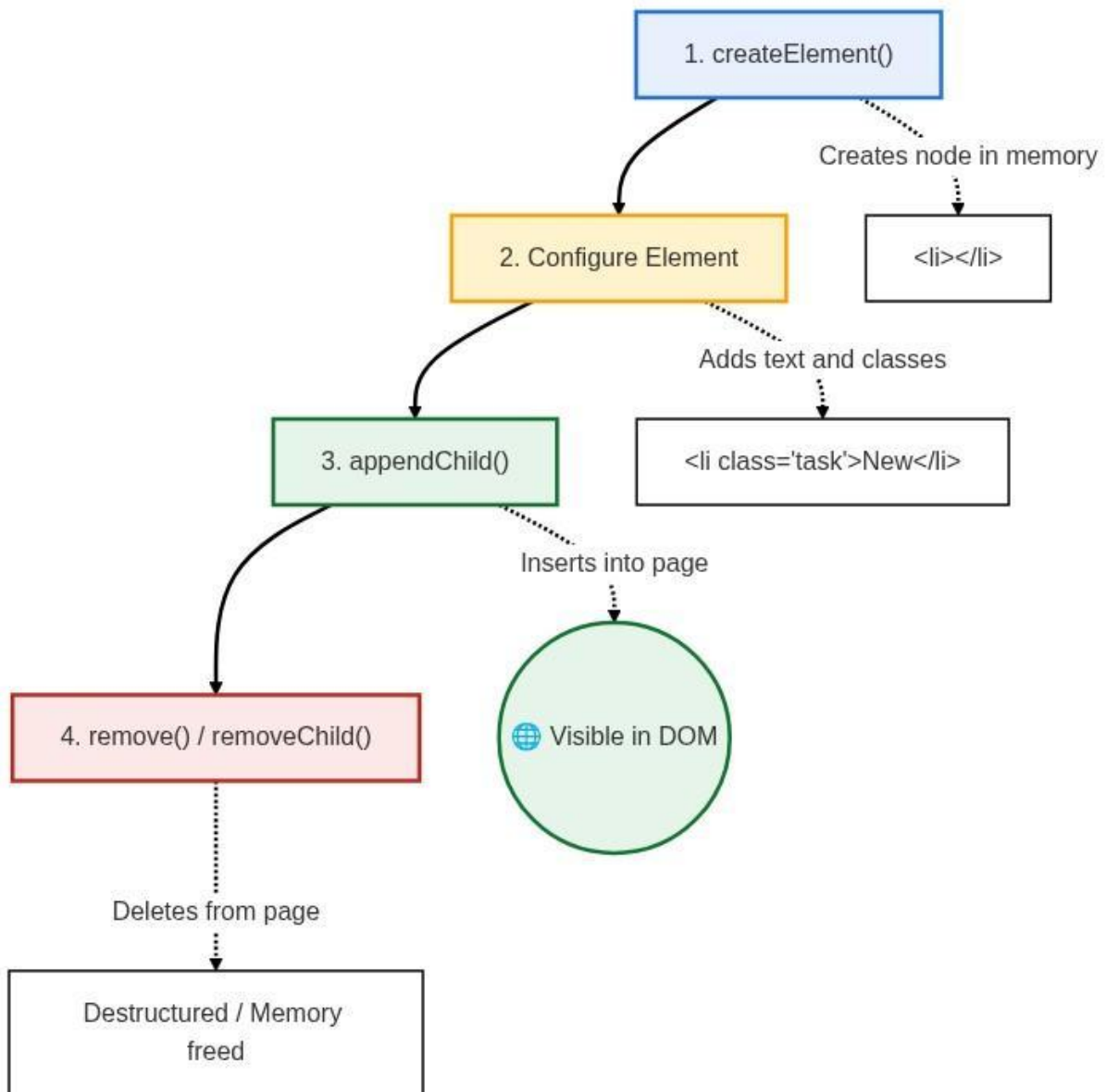


Fig. 4 - The diagram details the full lifecycle of an HTML node managed dynamically via JavaScript. The workflow spans four phases: **initial creation in RAM memory** (`createElement`), **attribute configuration** (CSS classes, inner text), **insertion into the visible web structure** (`appendChild`), and finally, **element removal from the graphic interface** (`remove` or `removeChild`). This last step breaks the element's connection to the DOM tree, allowing the browser to free up the associated memory resources.

4.3 The secret of interfaces: Event Bubbling

Imagine you have a delete button Inside a paragraph, which in turn is Inside a form.

What happens when you click the button?

In JavaScript, events "boil" and rise to the surface exactly like air bubbles in water (Event Bubbling).

When you click on the <button>, the browser triggers:

- Click on <button> (Child)
- Then Click on <p> (Parent)
- Then Click on <form> (Grandparent)
- Then Click on <body>... all the way to the Root!

```
// If you had listeners on each, a single click would trigger 3 actions!  
button.addEventListener('click', () => console.log("Click on button"));  
paragraph.addEventListener('click', () => console.log("Click on  
paragraph"));  
form.addEventListener('click', () => console.log("Click on form"));
```

4.4 Stopping propagation (stopPropagation)

Sometimes, developers use the `event.stopPropagation()` method to block the "bubble" from rising higher than the clicked element.

As a QA, you will hit this concept when you try to click a small icon (e.g., a checkmark) located inside a large box, but due to how propagation was handled on the Frontend, your click might only "hit" the parent, and the event never reaches the desired logic if you write the wrong selector.

4.5 Engineer Mindset in the AI Era

QA vs. AI: `Thread.sleep()` vs Smart Waiting

- **What AI does well:** If you ask an AI to write a test for a page where elements appear with a delay, it will write the click code for you.
- **The Trap (Where AI fails):** A poorly trained AI, when faced with a test that fails due to the latency described in section 4.1, will suggest the most toxic practice in QA: Static Waiting (Hard Sleep). It will generate a line of code saying `await page.waitForTimeout(3000);` (Stop the test completely for 3 seconds). If you have 200 tests and the AI inserts a 3-second pause in each, your test suite will run in 10 minutes instead of 30 seconds!
- **How a QA engineer thinks:** The engineer knows how `setTimeout` and the DOM

actually work. They will delete the fixed wait (the sleep) and make sure they use State-based waiting commands, asking Playwright to act dynamically: *"Wait only as long as necessary for the DOM to reach the correct state"*.

Chapter 5: Session 4 Assignment & Extra Practice (Towards Senior Level)

5.1 Homework Assignment (Session 4 Project)

Your Task (The Business Task):

In the previous chapters, we successfully added new tasks to the list. But a "To-Do" app is not complete if we can't delete our finished tasks! Your task is to modify the addition code so that each dynamically created task comes "packaged" with its own delete button, and that button actually works.

Execution note: You will write this code in your **app.js** file. Test it each time in the browser (Chrome) by refreshing the `index.html` page.

Technical Requirements (Acceptance Criteria for validation):

- **Selecting Initial Elements:**
 - Find the form (`#add-task-form`), the input (`#task-name`), and the list (`#active-tasks-list`) using `querySelector`.
- **Complex Dynamic Creation:**
 - When the form receives the submit event, cancel the native behavior (`preventDefault`).
 - Create a new `<li>` element and add the text from the input to it.
 - **(NEW)** Create another dynamic element: a `<button>`. Give it the text "Delete", the class `"delete-btn"`, and a `data-testid="delete-dynamic-task"` attribute.
- **Event Listener Inside Another Event:**
 - Right there, before injecting the button into the page, attach a 'click' event listener to it (the delete button created above).
 - When clicked, use the `.remove()` method to delete the parent `<li>` element.
- **Assembly (Lego):**
 - Take the delete button and "hook" it (with `appendChild`) inside the `<li>` element.
 - Take the `<li>` element (which now contains both the text and the button) and hook it into the main list (`<ul>`).

- Clear the input (value = "").

(QA Analogy: This scenario represents the ultimate End-to-End test: Create & Delete. In Playwright, you will fill the field, click 'Save', validate with expect that the task appeared, then click its 'Delete' button and validate with expect(locator).not.toBeVisible() that it disappeared!)

5.2 Bonus Challenge (Optional): Smart Button Disabling

The Scenario:

In top modern applications (Enterprise), the "Save" button is visually disabled (grey, unclickable) as long as the text field is empty, to prevent spam or sending invisible tasks.

Your Mission:

You need to add a reactive behavior. Find the "Save Task" button in the HTML. Find the input. Put an 'input' type addEventListener on the text field (this triggers on every letter typed by the user).

Write logic so that:

- If the input value is an empty string (""), the button receives the blocking attribute (using button.disabled = true;).
- If the input has at least one character, the button becomes active (button.disabled = false;).

5.3 Solution Keys (Answers & Explanations)

 **Don't cheat!** Read this only after you get stuck or try to write the code yourself for at least 20 minutes.

Homework Solution:

```
const form = document.querySelector('#add-task-form');
const taskInput = document.querySelector('#task-name');
const taskList = document.querySelector('#active-tasks-list');

form.addEventListener('submit', (event) => {
  event.preventDefault();
  const newText = taskInput.value;
```

```
// 1. Create the list element
const newLi = document.createElement('li');
newLi.className = 'task-item';
newLi.textContent = newText + " "; // Added a space before the
button

// 2. Create the delete button
const deleteBtn = document.createElement('button');
deleteBtn.textContent = 'Delete';
deleteBtn.className = 'delete-btn';
deleteBtn.setAttribute('data-testid', 'delete-dynamic-task');

// 3. Give the button the "power" to destroy its parent (the <li>)
deleteBtn.addEventListener('click', () => {
    newLi.remove();
    console.log("Task deleted from DOM!");
});

// 4. Assemble the pieces and inject them on screen
newLi.appendChild(deleteBtn);
taskList.appendChild(newLi);

// 5. Clear the field
taskInput.value = '';
});
```

✅ Bonus Challenge Solution (Live Validation):

```
// Find the submit button we want to manipulate
const submitBtn = document.querySelector('#add-task-btn');

// Set the initial state: when opening the page, the field is empty, so
we disable the button
submitBtn.disabled = true;

// Listen to every letter the user types in the input
taskInput.addEventListener('input', () => {
    // If text length is 0 (it's empty), disable remains true
    if (taskInput.value.length === 0) {
        submitBtn.disabled = true;
    } else {
        // Otherwise, if they wrote something, activate the button
        submitBtn.disabled = false;
    }
});
```



🎉 **Massive congratulations!** By solving this assignment, you have successfully completed the Frontend and JavaScript module (Sessions 1 - 4). You now hold the "keys" to the web interface. You know how pages are built (HTML/CSS), how they think (JS), and how elements interact with each other.

Your next stop? We will take the step towards **Asynchronism** and towards **Playwright!** You will take control, replacing your mouse with automation scripts that will execute these exact actions at the speed of light!